

Ship Fast

E-Commerce Real-Time Order Match Core

Name: Dharmit Dinesh Sathvara

Roll Number: 150096725080

Institution: ITM Skills University

Subject: Data Structures & Algorithms with C++

Batch: 2025-29

GitHub: <https://github.com/dharmitsathvara29-max/E-Commerce-Real-Time-Order-Match-Core>

ShipFast is the order processing backend for a large e-commerce marketplace handling thousands of orders per second.

The existing system suffers from several challenges:

- Product inventory is too large for efficient searching.
- Incorrect tax calculations cannot be rolled back.
- Orders arrive rapidly but are not processed efficiently.
- No mechanism exists to prioritize urgent shipments.
- Warehouses and transport links are not modeled.
- No shortest path calculation exists for logistics planning.
- Orders containing multiple items cannot be split intelligently across warehouses.

The objective is to design and implement a scalable order fulfillment system using Data Structures and

Algorithms in C++.

Objectives

1. Maintain a searchable inventory database.
2. Support rollback of tax calculations using a stack.
3. Process customer orders in FIFO checkout order.
4. Lookup orders quickly using a unique order ID.
5. Prioritize shipping using order urgency.
6. Model warehouse connectivity using a weighted graph.
7. Calculate shortest logistics paths using Dijkstra's Algorithm.
8. Split orders across warehouses to minimize delivery cost.

Design / Architecture

The system is built around a central ShipFastManager which coordinates inventory, orders, taxation, warehouse allocation and logistics planning.

Each requirement is implemented using an appropriate data structure to maximize efficiency and scalability.

Core Components

- Order
Represents a customer purchase containing items, amount and shipping priority.
- TaxRecord
Stores original and taxed amounts for rollback.
- InventoryIndex
Maintains product stock availability across warehouses.
- Graph
Represents warehouse and transport connectivity.
- OrderSystem
Handles checkout queue, shipping priority queue and order lookup.

. Key Modules

- Inventory Module
Add and maintain warehouse stock.
- Checkout Module
Accept customer orders in FIFO order.
- Shipping Module
Prioritize EXPRESS and NEXT_DAY shipments.
- Tax Module
Calculate taxes and support rollback.
- Lookup Module
Search orders using confirmation number.
- Logistics Module
Maintain warehouse transport network.
- Route Optimization Module
Find shortest shipping routes using Dijkstra.
- Stock Splitter Module
Allocate items from optimal warehouses.

Algorithm Description

1. Inventory List – map (B+ Tree Simulation)

Inventory is stored using a sorted map indexed by product name.

This simulates the ordered indexing behavior of disk-based B+ Trees commonly used in large e-commerce databases.

2. Tax Undo – Stack

Each tax calculation pushes a TaxRecord onto a stack.

Rollback pops the most recent transaction and restores the original amount.

3. Checkout Line – Queue

Orders are inserted into a queue and processed in FIFO order.

This guarantees fairness and preserves customer submission order.

4. Order Lookup – map

Orders are stored in a map indexed by Order ID.

This enables efficient retrieval of order information.

5. Shipping Sorter – Priority Queue

Orders are ranked by urgency.

NEXT_DAY orders receive highest priority, followed by EXPRESS and STANDARD.

6. Logistics Map – Graph

Warehouses are represented as graph nodes.

Transport routes are represented as weighted edges.

7. Fastest Paperwork – Dijkstra Algorithm

A min-heap priority queue implements Dijkstra's Algorithm to calculate minimum transport cost between warehouses.

8. Stock Splitter – Greedy Selection

Items are assigned to the warehouse with the minimum delivery cost while maintaining stock availability.

Orders containing multiple items can be fulfilled from multiple warehouses.

Complexity Analysis

1. Inventory Add/Search

Time: $O(\log n)$

Space: $O(n)$

2. Tax Calculation

Time: $O(1)$

Space: $O(t)$

3. Tax Rollback

Time: $O(1)$

Space: $O(1)$

4. Checkout Queue Insert

Time: $O(1)$

Space: $O(q)$

5. Order Lookup

Time: $O(\log n)$

Space: $O(n)$

6. Shipping Priority Insert

Time: $O(\log n)$

Space: $O(n)$

7. Add Logistics Route

Time: $O(1)$

Space: $O(E)$

8. Dijkstra Shortest Path

Time: $O((V+E) \log V)$

Space: $O(V+E)$

9. Stock Allocation

Time: $O(w)$

Space: $O(1)$

Where:

n = orders

q = queue size

t = tax history

V = warehouses

E = routes

w = warehouses containing stock

```

82         if(index.find(item) == index.end())
83             return false;
84
85         for(auto &p : index[item])
86         {
87             if(p.first == warehouse &&
88                p.second >= qty)
89             {
90                 p.second -= qty;
91                 return true;
92             }
93         }
94
95         return false;
96     }
97
98     void displayInventory()
99     {
100         cout << "\n===== \n";
101         cout << "FINAL INVENTORY REPORT \n";
102         cout << "===== \n";
103
104         map<int, vector<pair<string, int>>> warehouseView;
105
106         for(auto &itemEntry : index)
107         {
108             for(auto &warehouseEntry :
109                itemEntry.second)
110             {
111                 warehouseView[warehouseEntry.first]
112                     .push_back({
113                         itemEntry.first,
114                         warehouseEntry.second
115                     });
116             }
117         }
118
119         for(auto &w : warehouseView)
120         {
121             cout << "\nWarehouse "
122                  << w.first
123                  << endl;
124
125             for(auto &item : w.second)
126             {
127                 cout << " "
128                      << item.first
129                      << " : "
130                      << item.second
131                      << endl;
132             }
133         }
134     }
135 };
136
137 /* =====
138    |   |   |   |   |   TAX SYSTEM
139    ===== */
140
141 struct TaxRecord {
142     string orderId;
143     double originalAmount;
144     double finalAmount;
145 };
146
147 class TaxSystem {
148 public:
149
150     stack<TaxRecord> history;
151

```

```
/* =====  
| | | | | INVENTORY INDEX  
===== */
```

```
class InventoryIndex {  
public:
```

```
    struct std::__1::pair<int, int>
```

❖ Generate Copilot summary

```
    // item -> [(warehouse, qty)]  
    map<string, vector<pair<int, int>>> index;
```

```
    void addStock(int warehouse,  
                  const string &item,  
                  int qty)  
    {  
        index[item].push_back({warehouse, qty});  
    }
```

```
    int getRemainingStock(int warehouse,  
                           const string &item)  
    {  
        if(index.find(item) == index.end())  
            return 0;  
  
        for(auto &p : index[item])  
        {  
            if(p.first == warehouse)  
                return p.second;  
        }  
  
        return 0;  
    }
```

```
    bool reduceStock(int warehouse,  
                     const string &item,  
                     int qty)  
    {  
        if(index.find(item) == index.end())  
            return false;
```



```

vector<int> dijkstra(int src,
                    int n)
{
    vector<int> dist(n, INT_MAX);

    priority_queue<
        pair<int,int>,
        vector<pair<int,int>>,
        greater<pair<int,int>>
    > pq;

    dist[src] = 0;

    pq.push({0, src});

    while(!pq.empty())
    {
        auto current = pq.top();
        pq.pop();

        int d = current.first;
        int node = current.second;

        if(d > dist[node])
            continue;

        for(auto &nbr : adj[node])
        {
            int nextNode = nbr.first;
            int weight = nbr.second;

            if(dist[nextNode] >
               dist[node] + weight)
            {
                dist[nextNode] =
                    dist[node] + weight;

                pq.push({
                    dist[nextNode],
                    nextNode
                });
            }
        }
    }

    return dist;
}

```

```

=====
ORDER SYSTEM
=====

```

```

*/

```

```

struct CompareOrder {

    bool operator()(const Order &a,
                    const Order &b) const
    {
        return a.priority < b.priority;
    }
}

```

```

class OrderSystem {
public:
    queue<Order> checkoutQueue;

    priority_queue<
        Order,
        vector<Order>,
        CompareOrder
    > shippingQueue;
}

```

```

282     CompareOrder
283 > shippingQueue;
284
285     map<string, Order> orderDatabase;
286
287     void addOrder(const Order &o)
288     {
289         checkoutQueue.push(o);
290         orderDatabase[o.id] = o;
291     }
292 void prepareShipping()
293 {
294     while(!checkoutQueue.empty())
295     {
296         shippingQueue.push(
297             checkoutQueue.front()
298         );
299
300         checkoutQueue.pop();
301     }
302 }
303 bool shippingEmpty()
304 {
305     return shippingQueue.empty();
306 }
307 Order nextShippingOrder()
308 {
309     Order o =
310         shippingQueue.top();
311
312     shippingQueue.pop();
313
314     return o;
315 }
316
317 void lookupOrder(const string &id)
318 {
319     cout << "\n===== ORDER LOOKUP =====\n";
320
321     if(orderDatabase.find(id)
322        == orderDatabase.end())
323     {
324         cout << "Order not found.\n";
325         return;
326     }
327
328     Order o =
329         orderDatabase[id];
330
331     cout << "Order ID : "
332          << o.id
333          << endl;
334
335     cout << "Type      : "
336          << o.type
337          << endl;
338
339     cout << "Amount   : "
340          << o.amount
341          << endl;
342
343     cout << "Items    : ";
344
345     for(auto &item : o.items)
346     {
347         cout << item << " ";
348     }
349
350     cout << endl;
351 }
352 };

```

```

53  /* =====
54  |         |         |         |         |         |
55  |         |         |         |         |         | FULFILLMENT ENGINE
56  |         |         |         |         |         |
57  ===== */
58
59  class FulfillmentEngine {
60  private:
61      InventoryIndex &inventory;
62      Graph &graph;
63
64  public:
65      class InventoryIndex
66      {
67          ✦ Generate Copilot summary
68          InventoryIndex &inv,
69          Graph &g
70          : inventory(inv), graph(g) {}
71
72      int bestWarehouse(
73          const string &item,
74          const vector<int> &dist
75      )
76      {
77          if(inventory.index.find(item)
78             == inventory.index.end())
79              return -1;
80
81          int best = -1;
82          int bestCost = INT_MAX;
83
84          cout << "\nEvaluating Item: "
85               << item
86               << endl;
87
88          for(auto &entry :
89              inventory.index[item])
90          {
91              int warehouse =
92                  entry.first;
93
94              int stock =
95                  entry.second;
96
97              if(stock <= 0)
98                  continue;
99
100             int distanceCost =
101                 dist[warehouse];
102
103             int stockPenalty =
104                 max(0, 10 - stock);
105
106             int totalCost =
107                 distanceCost +
108                 stockPenalty;
109
110             cout
111                 << "Warehouse "
112                 << warehouse
113                 << " | Distance="
114                 << distanceCost
115                 << " | Stock="
116                 << stock
117                 << " | Penalty="
118                 << stockPenalty
119                 << " | TotalCost="
120                 << totalCost
121                 << endl;
122
123             if(totalCost < bestCost)
124             {
125                 bestCost = totalCost;
126             }
127         }
128     }
129
130     int bestWarehouse(
131         const string &item,
132         const vector<int> &dist
133     )
134     {
135         return bestWarehouse(item, dist);
136     }
137
138     void printBestWarehouse(
139         const string &item,
140         const vector<int> &dist
141     )
142     {
143         int bestWarehouse = bestWarehouse(item, dist);
144         int bestCost = bestCost(item, dist);
145         cout << "Best Warehouse: "
146              << bestWarehouse
147              << " | Best Cost: "
148              << bestCost
149              << endl;
150     }
151
152     void printInventoryIndex(
153         const InventoryIndex &inv,
154         const Graph &g
155     )
156     {
157         for(auto &entry : inv.index)
158         {
159             cout << "Item: "
160                  << entry.first
161                  << " | Entries: ";
162             for(auto &entry2 : entry.second)
163             {
164                 cout << "Warehouse: "
165                      << entry2.first
166                      << " | Stock: "
167                      << entry2.second
168                      << " | ";
169             }
170             cout << endl;
171         }
172     }
173
174     void printGraph(
175         const Graph &g
176     )
177     {
178         g.print();
179     }
180
181     void printInventory(
182         const InventoryIndex &inv,
183         const Graph &g
184     )
185     {
186         printInventoryIndex(inv, g);
187     }
188
189     void printBestWarehouse(
190         const string &item,
191         const vector<int> &dist
192     )
193     {
194         printBestWarehouse(item, dist);
195     }
196
197     void printInventoryIndex(
198         const InventoryIndex &inv,
199         const Graph &g
200     )
201     {
202         printInventoryIndex(inv, g);
203     }
204
205     void printGraph(
206         const Graph &g
207     )
208     {
209         printGraph(g);
210     }
211
212     void printInventory(
213         const InventoryIndex &inv,
214         const Graph &g
215     )
216     {
217         printInventory(inv, g);
218     }
219
220     void printBestWarehouse(
221         const string &item,
222         const vector<int> &dist
223     )
224     {
225         printBestWarehouse(item, dist);
226     }
227
228     void printInventoryIndex(
229         const InventoryIndex &inv,
230         const Graph &g
231     )
232     {
233         printInventoryIndex(inv, g);
234     }
235
236     void printGraph(
237         const Graph &g
238     )
239     {
240         printGraph(g);
241     }
242
243     void printInventory(
244         const InventoryIndex &inv,
245         const Graph &g
246     )
247     {
248         printInventory(inv, g);
249     }
250
251     void printBestWarehouse(
252         const string &item,
253         const vector<int> &dist
254     )
255     {
256         printBestWarehouse(item, dist);
257     }
258
259     void printInventoryIndex(
260         const InventoryIndex &inv,
261         const Graph &g
262     )
263     {
264         printInventoryIndex(inv, g);
265     }
266
267     void printGraph(
268         const Graph &g
269     )
270     {
271         printGraph(g);
272     }
273
274     void printInventory(
275         const InventoryIndex &inv,
276         const Graph &g
277     )
278     {
279         printInventory(inv, g);
280     }
281
282     void printBestWarehouse(
283         const string &item,
284         const vector<int> &dist
285     )
286     {
287         printBestWarehouse(item, dist);
288     }
289
290     void printInventoryIndex(
291         const InventoryIndex &inv,
292         const Graph &g
293     )
294     {
295         printInventoryIndex(inv, g);
296     }
297
298     void printGraph(
299         const Graph &g
300     )
301     {
302         printGraph(g);
303     }
304
305     void printInventory(
306         const InventoryIndex &inv,
307         const Graph &g
308     )
309     {
310         printInventory(inv, g);
311     }
312
313     void printBestWarehouse(
314         const string &item,
315         const vector<int> &dist
316     )
317     {
318         printBestWarehouse(item, dist);
319     }
320
321     void printInventoryIndex(
322         const InventoryIndex &inv,
323         const Graph &g
324     )
325     {
326         printInventoryIndex(inv, g);
327     }
328
329     void printGraph(
330         const Graph &g
331     )
332     {
333         printGraph(g);
334     }
335
336     void printInventory(
337         const InventoryIndex &inv,
338         const Graph &g
339     )
340     {
341         printInventory(inv, g);
342     }
343
344     void printBestWarehouse(
345         const string &item,
346         const vector<int> &dist
347     )
348     {
349         printBestWarehouse(item, dist);
350     }
351
352     void printInventoryIndex(
353         const InventoryIndex &inv,
354         const Graph &g
355     )
356     {
357         printInventoryIndex(inv, g);
358     }
359
360     void printGraph(
361         const Graph &g
362     )
363     {
364         printGraph(g);
365     }
366
367     void printInventory(
368         const InventoryIndex &inv,
369         const Graph &g
370     )
371     {
372         printInventory(inv, g);
373     }
374
375     void printBestWarehouse(
376         const string &item,
377         const vector<int> &dist
378     )
379     {
380         printBestWarehouse(item, dist);
381     }
382
383     void printInventoryIndex(
384         const InventoryIndex &inv,
385         const Graph &g
386     )
387     {
388         printInventoryIndex(inv, g);
389     }
390
391     void printGraph(
392         const Graph &g
393     )
394     {
395         printGraph(g);
396     }
397
398     void printInventory(
399         const InventoryIndex &inv,
400         const Graph &g
401     )
402     {
403         printInventory(inv, g);
404     }
405
406     void printBestWarehouse(
407         const string &item,
408         const vector<int> &dist
409     )
410     {
411         printBestWarehouse(item, dist);
412     }
413
414     void printInventoryIndex(
415         const InventoryIndex &inv,
416         const Graph &g
417     )
418     {
419         printInventoryIndex(inv, g);
420     }
421
422     void printGraph(
423         const Graph &g
424     )
425     {
426         printGraph(g);
427     }
428
429     void printInventory(
430         const InventoryIndex &inv,
431         const Graph &g
432     )
433     {
434         printInventory(inv, g);
435     }
436
437     void printBestWarehouse(
438         const string &item,
439         const vector<int> &dist
440     )
441     {
442         printBestWarehouse(item, dist);
443     }
444
445     void printInventoryIndex(
446         const InventoryIndex &inv,
447         const Graph &g
448     )
449     {
450         printInventoryIndex(inv, g);
451     }
452
453     void printGraph(
454         const Graph &g
455     )
456     {
457         printGraph(g);
458     }
459
460     void printInventory(
461         const InventoryIndex &inv,
462         const Graph &g
463     )
464     {
465         printInventory(inv, g);
466     }
467
468     void printBestWarehouse(
469         const string &item,
470         const vector<int> &dist
471     )
472     {
473         printBestWarehouse(item, dist);
474     }
475
476     void printInventoryIndex(
477         const InventoryIndex &inv,
478         const Graph &g
479     )
480     {
481         printInventoryIndex(inv, g);
482     }
483
484     void printGraph(
485         const Graph &g
486     )
487     {
488         printGraph(g);
489     }
490
491     void printInventory(
492         const InventoryIndex &inv,
493         const Graph &g
494     )
495     {
496         printInventory(inv, g);
497     }
498
499     void printBestWarehouse(
500         const string &item,
501         const vector<int> &dist
502     )
503     {
504         printBestWarehouse(item, dist);
505     }
506
507     void printInventoryIndex(
508         const InventoryIndex &inv,
509         const Graph &g
510     )
511     {
512         printInventoryIndex(inv, g);
513     }
514
515     void printGraph(
516         const Graph &g
517     )
518     {
519         printGraph(g);
520     }
521
522     void printInventory(
523         const InventoryIndex &inv,
524         const Graph &g
525     )
526     {
527         printInventory(inv, g);
528     }
529
530     void printBestWarehouse(
531         const string &item,
532         const vector<int> &dist
533     )
534     {
535         printBestWarehouse(item, dist);
536     }
537
538     void printInventoryIndex(
539         const InventoryIndex &inv,
540         const Graph &g
541     )
542     {
543         printInventoryIndex(inv, g);
544     }
545
546     void printGraph(
547         const Graph &g
548     )
549     {
550         printGraph(g);
551     }
552
553     void printInventory(
554         const InventoryIndex &inv,
555         const Graph &g
556     )
557     {
558         printInventory(inv, g);
559     }
560
561     void printBestWarehouse(
562         const string &item,
563         const vector<int> &dist
564     )
565     {
566         printBestWarehouse(item, dist);
567     }
568
569     void printInventoryIndex(
570         const InventoryIndex &inv,
571         const Graph &g
572     )
573     {
574         printInventoryIndex(inv, g);
575     }
576
577     void printGraph(
578         const Graph &g
579     )
580     {
581         printGraph(g);
582     }
583
584     void printInventory(
585         const InventoryIndex &inv,
586         const Graph &g
587     )
588     {
589         printInventory(inv, g);
590     }
591
592     void printBestWarehouse(
593         const string &item,
594         const vector<int> &dist
595     )
596     {
597         printBestWarehouse(item, dist);
598     }
599
600     void printInventoryIndex(
601         const InventoryIndex &inv,
602         const Graph &g
603     )
604     {
605         printInventoryIndex(inv, g);
606     }
607
608     void printGraph(
609         const Graph &g
610     )
611     {
612         printGraph(g);
613     }
614
615     void printInventory(
616         const InventoryIndex &inv,
617         const Graph &g
618     )
619     {
620         printInventory(inv, g);
621     }
622
623     void printBestWarehouse(
624         const string &item,
625         const vector<int> &dist
626     )
627     {
628         printBestWarehouse(item, dist);
629     }
630
631     void printInventoryIndex(
632         const InventoryIndex &inv,
633         const Graph &g
634     )
635     {
636         printInventoryIndex(inv, g);
637     }
638
639     void printGraph(
640         const Graph &g
641     )
642     {
643         printGraph(g);
644     }
645
646     void printInventory(
647         const InventoryIndex &inv,
648         const Graph &g
649     )
650     {
651         printInventory(inv, g);
652     }
653
654     void printBestWarehouse(
655         const string &item,
656         const vector<int> &dist
657     )
658     {
659         printBestWarehouse(item, dist);
660     }
661
662     void printInventoryIndex(
663         const InventoryIndex &inv,
664         const Graph &g
665     )
666     {
667         printInventoryIndex(inv, g);
668     }
669
670     void printGraph(
671         const Graph &g
672     )
673     {
674         printGraph(g);
675     }
676
677     void printInventory(
678         const InventoryIndex &inv,
679         const Graph &g
680     )
681     {
682         printInventory(inv, g);
683     }
684
685     void printBestWarehouse(
686         const string &item,
687         const vector<int> &dist
688     )
689     {
690         printBestWarehouse(item, dist);
691     }
692
693     void printInventoryIndex(
694         const InventoryIndex &inv,
695         const Graph &g
696     )
697     {
698         printInventoryIndex(inv, g);
699     }
700
701     void printGraph(
702         const Graph &g
703     )
704     {
705         printGraph(g);
706     }
707
708     void printInventory(
709         const InventoryIndex &inv,
710         const Graph &g
711     )
712     {
713         printInventory(inv, g);
714     }
715
716     void printBestWarehouse(
717         const string &item,
718         const vector<int> &dist
719     )
720     {
721         printBestWarehouse(item, dist);
722     }
723
724     void printInventoryIndex(
725         const InventoryIndex &inv,
726         const Graph &g
727     )
728     {
729         printInventoryIndex(inv, g);
730     }
731
732     void printGraph(
733         const Graph &g
734     )
735     {
736         printGraph(g);
737     }
738
739     void printInventory(
740         const InventoryIndex &inv,
741         const Graph &g
742     )
743     {
744         printInventory(inv, g);
745     }
746
747     void printBestWarehouse(
748         const string &item,
749         const vector<int> &dist
750     )
751     {
752         printBestWarehouse(item, dist);
753     }
754
755     void printInventoryIndex(
756         const InventoryIndex &inv,
757         const Graph &g
758     )
759     {
760         printInventoryIndex(inv, g);
761     }
762
763     void printGraph(
764         const Graph &g
765     )
766     {
767         printGraph(g);
768     }
769
770     void printInventory(
771         const InventoryIndex &inv,
772         const Graph &g
773     )
774     {
775         printInventory(inv, g);
776     }
777
778     void printBestWarehouse(
779         const string &item,
780         const vector<int> &dist
781     )
782     {
783         printBestWarehouse(item, dist);
784     }
785
786     void printInventoryIndex(
787         const InventoryIndex &inv,
788         const Graph &g
789     )
790     {
791         printInventoryIndex(inv, g);
792     }
793
794     void printGraph(
795         const Graph &g
796     )
797     {
798         printGraph(g);
799     }
800
801     void printInventory(
802         const InventoryIndex &inv,
803         const Graph &g
804     )
805     {
806         printInventory(inv, g);
807     }
808
809     void printBestWarehouse(
810         const string &item,
811         const vector<int> &dist
812     )
813     {
814         printBestWarehouse(item, dist);
815     }
816
817     void printInventoryIndex(
818         const InventoryIndex &inv,
819         const Graph &g
820     )
821     {
822         printInventoryIndex(inv, g);
823     }
824
825     void printGraph(
826         const Graph &g
827     )
828     {
829         printGraph(g);
830     }
831
832     void printInventory(
833         const InventoryIndex &inv,
834         const Graph &g
835     )
836     {
837         printInventory(inv, g);
838     }
839
840     void printBestWarehouse(
841         const string &item,
842         const vector<int> &dist
843     )
844     {
845         printBestWarehouse(item, dist);
846     }
847
848     void printInventoryIndex(
849         const InventoryIndex &inv,
850         const Graph &g
851     )
852     {
853         printInventoryIndex(inv, g);
854     }
855
856     void printGraph(
857         const Graph &g
858     )
859     {
860         printGraph(g);
861     }
862
863     void printInventory(
864         const InventoryIndex &inv,
865         const Graph &g
866     )
867     {
868         printInventory(inv, g);
869     }
870
871     void printBestWarehouse(
872         const string &item,
873         const vector<int> &dist
874     )
875     {
876         printBestWarehouse(item, dist);
877     }
878
879     void printInventoryIndex(
880         const InventoryIndex &inv,
881         const Graph &g
882     )
883     {
884         printInventoryIndex(inv, g);
885     }
886
887     void printGraph(
888         const Graph &g
889     )
890     {
891         printGraph(g);
892     }
893
894     void printInventory(
895         const InventoryIndex &inv,
896         const Graph &g
897     )
898     {
899         printInventory(inv, g);
900     }
901
902     void printBestWarehouse(
903         const string &item,
904         const vector<int> &dist
905     )
906     {
907         printBestWarehouse(item, dist);
908     }
909
910     void printInventoryIndex(
911         const InventoryIndex &inv,
912         const Graph &g
913     )
914     {
915         printInventoryIndex(inv, g);
916     }
917
918     void printGraph(
919         const Graph &g
920     )
921     {
922         printGraph(g);
923     }
924
925     void printInventory(
926         const InventoryIndex &inv,
927         const Graph &g
928     )
929     {
930         printInventory(inv, g);
931     }
932
933     void printBestWarehouse(
934         const string &item,
935         const vector<int> &dist
936     )
937     {
938         printBestWarehouse(item, dist);
939     }
940
941     void printInventoryIndex(
942         const InventoryIndex &inv,
943         const Graph &g
944     )
945     {
946         printInventoryIndex(inv, g);
947     }
948
949     void printGraph(
950         const Graph &g
951     )
952     {
953         printGraph(g);
954     }
955
956     void printInventory(
957         const InventoryIndex &inv,
958         const Graph &g
959     )
960     {
961         printInventory(inv, g);
962     }
963
964     void printBestWarehouse(
965         const string &item,
966         const vector<int> &dist
967     )
968     {
969         printBestWarehouse(item, dist);
970     }
971
972     void printInventoryIndex(
973         const InventoryIndex &inv,
974         const Graph &g
975     )
976     {
977         printInventoryIndex(inv, g);
978     }
979
980     void printGraph(
981         const Graph &g
982     )
983     {
984         printGraph(g);
985     }
986
987     void printInventory(
988         const InventoryIndex &inv,
989         const Graph &g
990     )
991     {
992         printInventory(inv, g);
993     }
994
995     void printBestWarehouse(
996         const string &item,
997         const vector<int> &dist
998     )
999     {
1000         printBestWarehouse(item, dist);
1001     }
1002
1003     void printInventoryIndex(
1004         const InventoryIndex &inv,
1005         const Graph &g
1006     )
1007     {
1008         printInventoryIndex(inv, g);
1009     }
1010
1011     void printGraph(
1012         const Graph &g
1013     )
1014     {
1015         printGraph(g);
1016     }
1017
1018     void printInventory(
1019         const InventoryIndex &inv,
1020         const Graph &g
1021     )
1022     {
1023         printInventory(inv, g);
1024     }
1025
1026     void printBestWarehouse(
1027         const string &item,
1028         const vector<int> &dist
1029     )
1030     {
1031         printBestWarehouse(item, dist);
1032     }
1033
1034     void printInventoryIndex(
1035         const InventoryIndex &inv,
1036         const Graph &g
1037     )
1038     {
1039         printInventoryIndex(inv, g);
1040     }
1041
1042     void printGraph(
1043         const Graph &g
1044     )
1045     {
1046         printGraph(g);
1047     }
1048
1049     void printInventory(
1050         const InventoryIndex &inv,
1051         const Graph &g
1052     )
1053     {
1054         printInventory(inv, g);
1055     }
1056
1057     void printBestWarehouse(
1058         const string &item,
1059         const vector<int> &dist
1060     )
1061     {
1062         printBestWarehouse(item, dist);
1063     }
1064
1065     void printInventoryIndex(
1066         const InventoryIndex &inv,
1067         const Graph &g
1068     )
1069     {
1070         printInventoryIndex(inv, g);
1071     }
1072
1073     void printGraph(
1074         const Graph &g
1075     )
1076     {
1077         printGraph(g);
1078     }
1079
1080     void printInventory(
1081         const InventoryIndex &inv,
1082         const Graph &g
1083     )
1084     {
1085         printInventory(inv, g);
1086     }
1087
1088     void printBestWarehouse(
1089         const string &item,
1090         const vector<int> &dist
1091     )
1092     {
1093         printBestWarehouse(item, dist);
1094     }
1095
1096     void printInventoryIndex(
1097         const InventoryIndex &inv,
1098         const Graph &g
1099     )
1100     {
1101         printInventoryIndex(inv, g);
1102     }
1103
1104     void printGraph(
1105         const Graph &g
1106     )
1107     {
1108         printGraph(g);
1109     }
1110
1111     void printInventory(
1112         const InventoryIndex &inv,
1113         const Graph &g
1114     )
1115     {
1116         printInventory(inv, g);
1117     }
1118
1119     void printBestWarehouse(
1120         const string &item,
1121         const vector<int> &dist
1122     )
1123     {
1124         printBestWarehouse(item, dist);
1125     }
1126
1127     void printInventoryIndex(
1128         const InventoryIndex &inv,
1129         const Graph &g
1130     )
1131     {
1132         printInventoryIndex(inv, g);
1133     }
1134
1135     void printGraph(
1136         const Graph &g
1137     )
1138     {
1139         printGraph(g);
1140     }
1141
1142     void printInventory(
1143         const InventoryIndex &inv,
1144         const Graph &g
1145     )
1146     {
1147         printInventory(inv, g);
1148     }
1149
1150     void printBestWarehouse(
1151         const string &item,
1152         const vector<int> &dist
1153     )
1154     {
1155         printBestWarehouse(item, dist);
1156     }
1157
1158     void printInventoryIndex(
1159         const InventoryIndex &inv,
1160         const Graph &g
1161     )
1162     {
1163         printInventoryIndex(inv, g);
1164     }
1165
1166     void printGraph(
1167         const Graph &g
1168     )
1169     {
1170         printGraph(g);
1171     }
1172
1173     void printInventory(
1174         const InventoryIndex &inv,
1175         const Graph &g
1176     )
1177     {
1178         printInventory(inv, g);
1179     }
1180
1181     void printBestWarehouse(
1182         const string &item,
1183         const vector<int> &dist
1184     )
1185     {
1186         printBestWarehouse(item, dist);
1187     }
1188
1189     void printInventoryIndex(
1190         const InventoryIndex &inv,
1191         const Graph &g
1192     )
1193     {
1194         printInventoryIndex(inv, g);
1195     }
1196
1197     void printGraph(
1198         const Graph &g
1199     )
1200     {
1201         printGraph(g);
1202     }
1203
1204     void printInventory(
1205         const InventoryIndex &inv,
1206         const Graph &g
1207     )
1208     {
1209         printInventory(inv, g);
1210     }
1211
1212     void printBestWarehouse(
1213         const string &item,
1214         const vector<int> &dist
1215     )
1216     {
1217         printBestWarehouse(item, dist);
1218     }
1219
1220     void printInventoryIndex(
1221         const InventoryIndex &inv,
1222         const Graph &g
1223     )
1224     {
1225         printInventoryIndex(inv, g);
1226     }
1227
1228     void printGraph(
1229         const Graph &g
1230     )
1231     {
1232         printGraph(g);
1233     }
1234
1235     void printInventory(
1236         const InventoryIndex &inv,
1237         const Graph &g
1238     )
1239     {
1240         printInventory(inv, g);
1241     }
1242
1243     void printBestWarehouse(
1244         const string &item,
1245         const vector<int> &dist
1246     )
1247     {
1248         printBestWarehouse(item, dist);
1249     }
1250
1251     void printInventoryIndex(
1252         const InventoryIndex &inv,
1253         const Graph &g
1254     )
1255     {
1256         printInventoryIndex(inv, g);
1257     }
1258
1259     void printGraph(
1260         const Graph &g
1261     )
1262     {
1263         printGraph(g);
1264     }
1265
1266     void printInventory(
1267         const InventoryIndex &inv,
1268         const Graph &g
1269     )
1270     {
1271         printInventory(inv, g);
1272     }
1273
1274     void printBestWarehouse(
1275         const string &item,
1276         const vector<int> &dist
1277     )
1278     {
1279         printBestWarehouse(item, dist);
1280     }
1281
1282     void printInventoryIndex(
1283         const InventoryIndex &inv,
1284         const Graph &g
1285     )
1286     {
1287         printInventoryIndex(inv, g);
1288     }
1289
1290     void printGraph(
1291         const Graph &g
1292     )
1293     {
1294         printGraph(g);
1295     }
1296
1297     void printInventory(
1298         const InventoryIndex &inv,
1299         const Graph &g
1300     )
1301     {
1302         printInventory(inv, g);
1303     }
1304
1305     void printBestWarehouse(
1306         const string &item,
1307         const vector<int> &dist
1308     )
1309     {
1310         printBestWarehouse(item, dist);
1311     }
1312
1313     void printInventoryIndex(
1314         const InventoryIndex &inv,
1315         const Graph &g
1316     )
1317     {
1318         printInventoryIndex(inv, g);
1319     }
1320
1321     void printGraph(
1322         const Graph &g
1323     )
1324     {
1325         printGraph(g);
1326     }
1327
1328     void printInventory(
1329         const InventoryIndex &inv,
1330         const Graph &g
1331     )
1332     {
1333         printInventory(inv, g);
1334     }
1335
1336     void printBestWarehouse(
1337         const string &item,
1338         const vector<int> &dist
1339     )
1340     {
1341         printBestWarehouse(item, dist);
1342     }
1343
1344     void printInventoryIndex(
1345         const InventoryIndex &inv,
1346         const Graph &g
1347     )
1348     {
1349         printInventoryIndex(inv, g);
1350     }
1351
1352     void printGraph(
1353         const Graph &g
1354     )
1355     {
1356         printGraph(g);
1357     }
1358
1359     void printInventory(
1360         const InventoryIndex &inv,
1361         const Graph &g
1362     )
1363     {
1364         printInventory(inv, g);
1365     }
1366
1367     void printBestWarehouse(
1368         const string &item,
1369         const vector<int> &dist
1370     )
1371     {
1372         printBestWarehouse(item, dist);
1373     }
1374
1375     void printInventoryIndex(
1376         const InventoryIndex &inv,
1377         const Graph &g
1378     )
1379     {
1380         printInventoryIndex(inv, g);
1381     }
1382
1383     void printGraph(
1384         const Graph &g
1385     )
1386     {
1387         printGraph(g);
1388     }
1389
1390     void printInventory(
1391         const InventoryIndex &inv,
1392         const Graph &g
1393     )
1394     {
1395         printInventory(inv, g);
1396     }
1397
1398     void printBestWarehouse(
1399         const string &item,
1400         const vector<int> &dist
1401     )
1402     {
1403         printBestWarehouse(item, dist);
1404     }
1405
1406     void printInventoryIndex(
1407         const InventoryIndex &inv,
1408         const Graph &g
1409     )
1410     {
1411         printInventoryIndex(inv, g);
1412     }
1413
1414     void printGraph(
1415         const Graph &g
1416     )
1417     {
1418         printGraph(g);
1419     }
1420
1421     void printInventory(
1422         const InventoryIndex &inv,
1423         const Graph &g
1424     )
1425     {
1426         printInventory(inv, g);
1427     }
1428
1429     void printBestWarehouse(
1430         const string &item,
1431         const vector<int> &dist
1432     )
1433     {
1434         printBestWarehouse(item, dist);
1435     }
1436
1437     void printInventoryIndex(
1438         const InventoryIndex &inv,
1439         const Graph &g
1440     )
1441     {
1442         printInventoryIndex(inv, g);
1443     }
1444
1445     void printGraph(
1446         const Graph &g
1447     )
1448     {
1449         printGraph(g);
1450     }
1451
1452     void printInventory(
1453         const InventoryIndex &inv,
1454         const Graph &g
1455     )
1456     {
1457         printInventory(inv, g);
1458     }
1459
1460     void printBestWarehouse(
1461         const string &item,
1462         const vector<int> &dist
1463     )
1464     {
1465         printBestWarehouse(item, dist);
1466     }
1467
1468     void printInventoryIndex(
1469         const InventoryIndex &
```

```

    }

    if(best != -1)
    {
        cout
            << "Selected Warehouse: "
            << best
            << endl;
    }

    return best;
}

void processOrder(
    const Order &order,
    const vector<int> &dist
)
{
    cout << "\n=====\\n";
    cout << "Processing Order: "
        << order.id
        << endl;

    cout << "Priority Level : "
        << order.priority
        << endl;

    cout << "=====\\n";

    for(auto &item :
        order.items)
    {
        int warehouse =
            bestWarehouse(
                item,
                dist
            );

        if(warehouse == -1)
        {
            cout
                << "Item "
                << item
                << " OUT OF STOCK"
                << endl;

            continue;
        }

        inventory.reduceStock(
            warehouse,
            item,
            1
        );

        cout
            << "Item: "
            << item
            << " | Warehouse: "
            << warehouse
            << " | Remaining Stock: "
            << inventory.getRemainingStock(
                warehouse,
                item
            )
            << endl;
    }
}

```



```

        cout << "Destination warehouse: ";
        cin >> v;

        cout << "Distance/Cost: ";
        cin >> w;

        logistics.addEdge(
            u,
            v,
            w
        );

        cout << "Route Added!\n";
        break;
    }

    case 3:
    {
        string id;
        string type;
        double amount;

        int n;

        vector<string> items;

        cout << "\nOrder ID: ";
        cin >> id;

        cout << "Order Type (EXPRESS/NEXT_DAY/STANDARD): ";
        cin >> type;

        cout << "Amount: ";
        cin >> amount;

        cout << "Number of Items: ";
        cin >> n;

        for(int i=0;i<n;i++)
        {
            string item;

            cout << "Item "
                << i+1
                << ": ";

            cin >> item;

            items.push_back(item);
        }

        orderSystem.addOrder(
            Order(
                id,
                items,
                type,
                amount
            )
        );

        cout << "Order Added!\n";

        break;
    }

    case 4:
    {

```

```

        cout << "\nEnter Order ID: ";
        cin >> id;

        orderSystem.lookupOrder(id);

        break;
    }

    case 5:
    {
        int totalWarehouses;

        cout << "\nEnter Number Of Warehouses: ";
        cin >> totalWarehouses;

        vector<int> dist =
            logistics.dijkstra(
                0,
                totalWarehouses
            );

        orderSystem.prepareShipping();
    }

while(
    !orderSystem.shippingEmpty()
)
{
    Order current =
        orderSystem.nextShippingOrder();

    double finalAmount =
        taxSystem.applyTax(
            current.id,
            current.amount
        );

    engine.processOrder(
        current,
        dist
    );

    cout
        << "Final Amount: "
        << finalAmount
        << endl;

    break;
}

    case 6:
    {
        taxSystem.rollback();
        break;
    }

    case 7:
    {
        inventory.displayInventory();
        break;
    }

    case 0:
    {
        cout << "\nExiting...\n";
        break;
    }

    default:
    {
        cout << "\nInvalid Choice!\n";
    }
}

```

```
=====
SHIPFAST MANAGEMENT SYSTEM
=====
```

1. Add Inventory
2. Add Logistics Route
3. Add Order
4. Order Lookup
5. Process Orders
6. Tax Rollback
7. Show Inventory
0. Exit

Enter Choice: 1

Warehouse ID: 0
Item Name: Laptop
Quantity: 10
Inventory Added!

```
=====
SHIPFAST MANAGEMENT SYSTEM
=====
```

1. Add Inventory
2. Add Logistics Route
3. Add Order
4. Order Lookup
5. Process Orders
6. Tax Rollback
7. Show Inventory
0. Exit

Enter Choice: 1

Warehouse ID: 1
Item Name: Mobile
Quantity: 40
Inventory Added!

```
=====
SHIPFAST MANAGEMENT SYSTEM
=====
```

1. Add Inventory
2. Add Logistics Route
3. Add Order
4. Order Lookup
5. Process Orders
6. Tax Rollback
7. Show Inventory
0. Exit

Enter Choice: 2

Source Warehouse: 0
Destination Warehouse: 1
Distance/Cost: 100
Route Added!

```
=====
SHIPFAST MANAGEMENT SYSTEM
=====
```

1. Add Inventory
2. Add Logistics Route
3. Add Order
4. Order Lookup
5. Process Orders
6. Tax Rollback
7. Show Inventory
0. Exit

Enter Choice: 2

Source Warehouse: 1
Destination Warehouse: 2
Distance/Cost: 200
Route Added!

Enter Choice: 4

Enter Order ID: 01

```
===== ORDER LOOKUP =====
```

Order ID : 01
Type : EXPRESS
Amount : 50000
Items : Mobile Earphones

```
Enter Choice: 5
```

```
Enter Number Of Warehouses: 3
```

```
=====
```

```
Processing Order: 01
```

```
Priority Level : 3
```

```
=====
```

```
Evaluating Item: Mobile
```

```
Warehouse 1 | Distance=100 | Stock=40 | Penalty=0 | TotalCost=100
```

```
Selected Warehouse: 1
```

```
Item: Mobile | Warehouse: 1 | Remaining Stock: 39
```

```
Evaluating Item: Earphones
```

```
Warehouse 2 | Distance=300 | Stock=30 | Penalty=0 | TotalCost=300
```

```
Selected Warehouse: 2
```

```
Item: Earphones | Warehouse: 2 | Remaining Stock: 29
```

```
Final Amount: 59000
```

Results and Findings

- Inventory indexing successfully handled warehouse stock.
- Stack-based rollback correctly restored previous tax calculations.
- FIFO queue preserved customer order submission sequence.
- Priority queue correctly ranked urgent shipments.
- Order lookup successfully retrieved orders by ID.
- Dijkstra's Algorithm correctly identified shortest logistics paths.
- Stock splitter allocated products across warehouses.
- The architecture mirrors real-world systems used by Amazon and Flipkart.

Conclusion

The ShipFast E-Commerce Real-Time Order Match Core demonstrates practical applications of Maps, Queues, Stacks, Priority Queues and Graphs in solving real-world order fulfillment problems.

Each data structure was selected based on its performance characteristics and suitability for the specific task.

The system is modular, scalable and reflects modern e-commerce fulfillment architectures used by Ama